

Women-Only Ride Hailing App PRD

Project Overview

Build a women-only ride-hailing platform inspired by Bolt and Uber, designed specifically for female riders and female drivers. The platform will prioritize safety, trust, emergency response systems, live tracking, and temporary dashcam evidence storage.

The application will include:

- Rider mobile app
- Driver mobile app
- Admin dashboard
- Real-time GPS tracking
- SOS emergency system
- Dashcam integration with temporary cloud storage
- Driver verification and onboarding
- Payments and ride management

Product Vision

Create the safest ride-hailing platform for women in Nigeria by combining:

- verified female drivers,
- real-time safety tools,
- emergency evidence systems,
- and trusted ride experiences.

The goal is not just transportation. The goal is safety, trust, and reliability.

Target Audience

Primary Users

- Women in Lagos and other major cities
- Students
- Working professionals
- Mothers and families

- Women traveling at night

Secondary Users

- Female drivers seeking flexible income
- Parents booking rides for daughters

Core MVP Features

Rider Features

Authentication

- Sign up
- Login
- OTP verification
- Password reset
- JWT authentication

Ride Booking

- Set pickup location
- Set destination
- View system suggested fare based on distance
- Propose a fare — rider can increase above suggested but never below the floor price
- Request ride
- Accept or reject driver counter-offer
- Cancel ride
- Ride status updates

Live Tracking

- Real-time vehicle tracking
- Driver ETA
- Share trip with trusted contacts
- Route visualization

Safety Features

- SOS emergency button
- Trip sharing

- Driver verification badge
- Report driver
- Emergency contact system

Payments

- Wallet
- Card payment
- Paystack integration
- Ride receipts

User Profile

- Ride history
- Saved locations
- Ratings and reviews
- Account settings

Driver Features

Authentication

- Driver registration
- Driver verification
- Upload documents
- Vehicle verification

Driver Dashboard

- Online/offline toggle
- View ride offers with rider proposed fare
- Accept rider proposed fare
- Counter-offer with a different fare
- Earnings dashboard
- Trip history

Navigation

- Navigate to rider
- Navigate to destination
- Real-time route updates

Safety

- Emergency reporting
- Dashcam integration
- Incident reporting

Ride Confirmation and Contact Feature

After a driver accepts a ride, both the rider and driver must be able to see each other's details for confirmation and communication purposes.

What the Rider Sees After Booking is Accepted

- Driver full name
- Driver profile photo
- Driver average rating
- Car make and model (e.g. Toyota Corolla)
- Car color
- Car plate number
- Driver phone number (to call directly from their phone)
- Real-time driver location on map
- Estimated time of arrival (ETA)
- Agreed fare amount

What the Driver Sees After Accepting a Ride

- Rider full name
- Rider profile photo (if available)
- Rider phone number (to call directly from their phone)
- Pickup location on map
- Destination address
- Agreed fare amount

Calling Approach - MVP vs Future

MVP (Now):

Both rider and driver phone numbers are displayed on their respective screens after a ride is accepted. They can call each other directly using their device native phone app.

Future Enhancement:

Replace direct number display with in-app calling using masked/anonymous numbers via Twilio or Africa's Talking. This protects user privacy while still allowing voice communication inside the app.

Fare Model - InDrive Style with Floor Protection

SafeHer uses a rider-proposed fare model inspired by InDrive. The system calculates a suggested fare based on distance and pricing config. The rider can increase the fare but cannot go below the floor price. This protects drivers from unfairly low offers while still giving riders control over what they pay.

How It Works

- Rider enters pickup and destination
- System calculates suggested fare from PricingConfig in database
- System calculates floor price (70% of suggested fare)
- Rider sees the suggested fare and the minimum they can offer
- Rider can increase the fare above suggested — no upper limit
- Rider CANNOT go below the floor price — enforced by the app and server
- All nearby eligible drivers see the ride offer and the rider proposed fare
- Driver accepts the fare — ride confirmed immediately
- Driver counter-offers with a higher amount — rider is notified
- Rider accepts the counter-offer — ride confirmed
- Rider rejects the counter-offer — other drivers can still accept the original offer
- No limit on driver counter-offer — market self-regulates
- Once any offer is accepted — all other pending offers are dismissed

Fare Calculation Formula

All values come from PricingConfig table — admin can change them without touching code:

- $\text{suggestedFare} = \text{baseFare} + (\text{distanceKm} \times \text{perKmRate})$
- $\text{floorPrice} = \text{suggestedFare} \times \text{floorPercent}$
- minimumFare applies if calculated fare is below the minimum threshold

Lagos Default Pricing (Admin Configurable)

- Base fare: NGN 1,000 flat fee for every ride
- Per km rate: NGN 250 per km
- Minimum fare: NGN 1,500 for very short trips

- Floor percent: 70% — rider cannot offer below this
- Platform fee: 10% commission taken from agreed fare

Example Calculations

- 3km trip: $\text{NGN } 1,000 + (3 \times 250) = \text{NGN } 1,750$ — floor: NGN 1,225
- 10km trip: $\text{NGN } 1,000 + (10 \times 250) = \text{NGN } 3,500$ — floor: NGN 2,450
- 20km trip: $\text{NGN } 1,000 + (20 \times 250) = \text{NGN } 6,000$ — floor: NGN 4,200

Why No Driver Cap

InDrive does not cap driver counter-offers. If a driver offers too high, the rider simply rejects and waits for another driver. The market self-regulates. SafeHer follows the same approach.

Future Enhancement

In a future version, when a driver counter-offers the app will show the rider the counter amount alongside two other suggested amounts with small differences to guide the rider's decision — similar to how InDrive handles counter-offers.

Ride Matching System

SafeHer uses a rider-proposed fare matching model inspired by InDrive. Riders set their own price within the allowed range and drivers choose which offers to accept.

Driver Eligibility

- Online (onlineStatus: true)
- Verified (verificationStatus: VERIFIED)
- Not currently on an active ride (no ride with status ACCEPTED, DRIVER_ARRIVED or TRIP_STARTED)
- Within 5km radius of the rider pickup location

Matching Flow - InDrive Model

- Rider submits a ride request with their proposed fare
- System broadcasts the offer to ALL eligible drivers within 5km
- Each driver sees the ride details and the rider proposed fare
- Driver accepts the fare — ride confirmed immediately

- Driver counter-offers with a higher fare — rider is notified
- Multiple drivers can counter-offer — rider chooses which to accept
- Once rider accepts any offer — other pending offers are dismissed

Driver Availability Rules

- Driver manually sets themselves online or offline via the toggle in their dashboard
- Driver is automatically unavailable when they have an active ride
- Driver becomes available again when a ride is completed or cancelled
- System never sends a new ride request to a driver who is on a trip

No Drivers Available - Search Expansion

- 0-30 seconds: Search within 5km — Looking for drivers near you...
- 30-60 seconds: Expand to 7km — Expanding search area...
- 60-90 seconds: Expand to 10km — Searching a wider area...

If no driver accepts or counter-offers after 90 seconds:

- Rider sees: No drivers available right now
- Option 1: Try Again — starts a fresh search immediately
- Option 2: Get Notified — app notifies rider when a driver becomes available nearby

Route Deviation Detection

The system monitors the driver GPS location against the expected route during an active trip.

- Rider receives an in-app alert: Your driver appears to be off route
- Admin dashboard receives an automatic alert for monitoring
- Rider emergency contacts are notified via SMS
- All dashcam video chunks for the ride are automatically locked as evidence

Admin Dashboard Features

User Management

- View riders
- View drivers
- Suspend accounts
- Verify drivers

Ride Management

- Monitor active rides
- View ride history
- Resolve disputes
- Emergency ride monitoring
- View fare negotiation history per ride

Pricing Management

- Update base fare, per km rate, minimum fare, floor percent and platform fee
- Changes apply immediately to all new ride requests without redeploying code
- Future: add city-based pricing rows for Abuja, Port Harcourt etc
- Future: add surge pricing during peak hours

Safety Management

- Access incident reports
- Review emergency footage
- Manage SOS alerts
- Monitor route deviation alerts

Analytics

- Total rides
- Revenue
- Active users
- Driver performance
- Ride heatmaps
- Average offered vs agreed fare — to understand market pricing

Safety Camera System

Objective

Create a temporary evidence storage system that improves rider safety without permanently storing large video files.

How It Works

Dashcam Recording

- Dashcam records during active rides
- Video split into encrypted chunks
- Clips uploaded periodically

Temporary Cloud Storage

- Videos stored temporarily for 12-24 hours
- Automatic deletion after expiration
- Encrypted storage

Emergency Evidence Preservation

If:

- SOS button is pressed
- suspicious route deviation occurs
- emergency is reported

Then:

- recent video footage is locked
- protected from deletion
- preserved as evidence

Access Rules

- Users can request temporary access
- Admin approval required for sensitive access
- Access logs stored securely
- Privacy-protected system

Technical Architecture

Tech Stack

Mobile Frontend

- React Native
- TypeScript

Backend

- Node.js

- Express.js

Database

- PostgreSQL

ORM

- Prisma ORM

Real-Time Communication

- Socket.IO
- WebSockets

Authentication

- JWT
- Refresh Tokens

Maps and Navigation

- Google Maps API
- Google Directions API
- Geocoding API

Payments

- Paystack

Push Notifications

- Firebase Cloud Messaging

Video Processing

- FFmpeg

Cloud Storage

- AWS S3 or Cloudflare R2

DevOps and Infrastructure

- Docker — containerization
- docker-compose — local development environment
- GitHub Actions — CI/CD pipeline

- Kubernetes — container orchestration and scaling
- Helm — Kubernetes deployment management
- Prometheus + Grafana — monitoring and dashboards
- Nginx Ingress — traffic routing
- Railway or Render — staging environment

Deployment

- Railway (staging)
- AWS (production)
- Docker

System Modules

Authentication Module

Handles:

- user registration
- login
- OTP verification
- token generation
- refresh tokens

Ride Matching Module

Handles:

- ride requests with rider proposed fare
- nearby driver search within 5km radius using Haversine formula
- broadcasting offer to all eligible drivers simultaneously
- fare negotiation — driver accept or counter-offer, rider accept or reject
- floor price enforcement — rider cannot offer below 70% of suggested fare
- automatic search expansion if no driver responds (5km to 7km to 10km over 90 seconds)
- driver availability checks (online + verified + no active ride)
- ride status updates

Fare Calculation Module

Handles:

- fetching active PricingConfig from database
- calculating suggested fare based on distance and pricing config
- calculating floor price (minimum rider can offer)
- validating rider offers against floor price
- calculating platform fee from agreed fare

GPS Tracking Module

Handles:

- real-time coordinates
- trip route tracking
- ETA calculations
- route deviation detection

Safety Module

Handles:

- SOS triggers
- emergency contacts
- live trip sharing
- incident reporting
- route deviation alerts to rider, admin and emergency contacts

Ride Confirmation and Contact Module

Handles:

- sending driver details (name, photo, car info, plate number, phone) to rider after acceptance
- sending rider details (name, phone, pickup) to driver after acceptance
- displaying agreed fare to both parties after negotiation
- displaying phone numbers on both apps for direct calling
- future: masked in-app calling via Twilio or Africa's Talking

Video Evidence Module

Handles:

- dashcam uploads
- encrypted storage
- temporary retention

- evidence locking
- automatic deletion

Payment Module

Handles:

- wallet funding
- card payments
- ride charges based on agreed fare
- platform fee deduction (10% of agreed fare)
- driver payouts (90% of agreed fare)

Database Design

Core Tables

Users

- id
- name
- email
- phone
- role
- created_at

Drivers

- id
- user_id
- vehicle_id
- verification_status
- online_status
- currentLat — current GPS latitude, null when offline
- currentLng — current GPS longitude, null when offline
- locationUpdatedAt — timestamp of last location update

Vehicles

- id
- plate_number

- model
- color
- verification_status

Rides

- id
- rider_id
- driver_id
- pickup_location
- destination
- status
- estimatedFare — system suggested fare based on distance and pricing config
- offeredFare — fare proposed by the rider
- counterFare — fare counter-offered by the driver if any
- agreedFare — final fare both parties agreed on
- fare — confirmed at trip completion, equals agreedFare
- started_at
- ended_at

PricingConfig

- id
- city
- baseFare
- perKmRate
- minimumFare
- floorPercent
- platformFeePercent
- isActive

RideLocations

- id
- ride_id
- latitude
- longitude
- timestamp

EmergencyReports

- id
- ride_id
- report_type
- created_at

VideoChunks

- id
- ride_id
- storage_url
- expires_at
- locked_status

Real-Time Features

Socket Events

Rider Events

- request_ride — includes proposed fare (must be at or above floor price)
- accept_counter — rider accepts driver counter-offer
- reject_counter — rider rejects driver counter-offer
- cancel_ride
- track_driver
- sos_triggered
- ride_accepted — receives driver details + car info + phone + agreed fare
- no_drivers_found — triggers search expansion status updates

Driver Events

- new_ride_offer — receives ride details and rider proposed fare
- accept_offer — driver accepts rider proposed fare
- counter_offer — driver proposes a higher fare
- update_location
- trip_started
- trip_completed
- ride_assigned — receives rider name, phone, pickup and agreed fare
- ride_no_longer_available — another driver was accepted first

Admin Events

- emergency_alert
- driver_flagged
- suspicious_route

API Structure

Authentication Routes

- POST /auth/register
- POST /auth/login
- POST /auth/refresh

Ride Routes

- POST /rides/request — includes offeredFare in body, server validates against floor
- PATCH /rides/:id/counter — driver submits counter-offer
- PATCH /rides/:id/accept-offer — rider accepts driver counter-offer
- POST /rides/cancel
- GET /rides/:id
- GET /rides/:id/driver-details — returns driver name, photo, car info, plate, phone
- GET /rides/:id/rider-details — returns rider name and phone (driver only)

Pricing Routes

- GET /pricing — fetch active pricing config (public)
- PATCH /pricing/:id — admin updates pricing config

Driver Routes

- POST /drivers/online
- POST /drivers/offline
- GET /drivers/profile

Emergency Routes

- POST /emergency/sos
- POST /emergency/report

Video Routes

- POST /videos/upload
- GET /videos/:rideId
- DELETE /videos/expired

Development Phases

Phase 1 - Core MVP

Build:

- authentication
- ride booking with InDrive fare model and floor price protection
- fare negotiation between rider and driver
- live tracking
- driver system
- admin dashboard
- ride confirmation screen with driver and car details, phone number and agreed fare

Goal: Validate ride flow.

Phase 2 - Safety Layer

Build:

- SOS system
- emergency contacts
- trip sharing
- route deviation alerts to rider, admin and emergency contacts

Goal: Validate safety features.

Phase 3 - Dashcam Integration

Build:

- dashcam uploads
- temporary storage
- auto deletion
- evidence locking

Goal: Validate evidence system.

Phase 4 - Payments and Scaling

Build:

- Paystack integration
- wallets
- driver payouts based on agreed fare
- optimization
- in-app calling with masked numbers via Twilio or Africa's Talking

Goal: Production readiness.

Phase 5 - DevOps and Infrastructure

Build:

- Docker containerization
- GitHub Actions CI/CD pipeline
- Kubernetes deployment and orchestration
- monitoring, logging and alerting
- production launch

Goal: Scalable, reliable, production-grade infrastructure.

Security Requirements

- Encrypt video uploads
- Use HTTPS everywhere
- Hash passwords using bcrypt
- Store JWT secrets securely
- Validate all API inputs
- Implement rate limiting
- Protect admin routes
- Log sensitive access
- Phone numbers only shown to matched rider and driver — never publicly exposed
- Driver eligibility verified server-side before every ride broadcast
- Floor price enforced server-side — client validation alone is not enough
- Agreed fare stored and used for payment — cannot be changed after acceptance

Non-Functional Requirements

Performance

- Real-time location updates under 3 seconds
- Fare negotiation response under 2 seconds
- Low mobile data usage

Scalability

- Modular backend structure
- Kubernetes auto-scaling for traffic spikes
- Queue system for heavy tasks
- Cloud-based storage

Reliability

- Retry failed uploads
- Recover interrupted rides
- Persistent socket reconnection
- Zero downtime deployments via Kubernetes rolling updates
- Graceful handling of no drivers available scenario

Future Features

- AI anomaly detection
- Voice emergency trigger
- Driver facial verification
- Subscription plans
- Scheduled rides
- Corporate accounts
- Multi-city support
- In-app calling with masked numbers via Twilio or Africa's Talking
- Suggested fare range hint shown to rider during booking
- Surge pricing during peak hours — admin configurable
- Smart dispatching

MVP Goal

The MVP goal is not to compete with Bolt immediately.

The MVP goal is:

- safe rides,
- reliable operations,
- trusted female driver network,
- and validated demand.

Focus on solving safety and trust first before scaling aggressively.